DC^{3}S: Telemetry-Reliability-Weighted Conformal Safety Shield for Battery Dispatch under IoT Sensor Faults

Pratik Niroula
MNSU CIT (Major), Math (Minor)

February 19, 2026

Abstract

DC^{3}S addresses battery dispatch under unreliable IoT telemetry by coupling telemetry-reliability scoring, conformal interval inflation, and safety-constrained action repair in one online loop. In the `drift_combo` CPSBench scenario, DC^{3}S raises mean PICP@90 to 38.3% from 33.8% for deterministic dispatch, while holding intervention rate at 0.00%. This calibration gain is achieved with wider mean intervals (2249.64 vs 666.98) and an expected cost of USD 61,310,572.02, which is USD 160,899.55 above `robust_fixed_interval`. The resulting tradeoff is explicit rather than hidden: when telemetry quality degrades, DC^{3}S buys coverage and eliminates observed violation events by widening uncertainty sets and repairing actions against safety constraints.

This paper presents DC^{3}S as a telemetry-reliability-weighted conformal safety shield for battery dispatch under sensor faults such as dropout, delay jitter, stale readings, out-of-order packets, and spikes. The method combines online fault scoring, drift-aware uncertainty inflation, dispatch optimization, and certificate-backed audit persistence within the GridPulse stack. The contribution is not a new battery objective alone, but a calibrated control wrapper that converts degraded telemetry into bounded uncertainty and executable safe actions. Publication-facing quantitative claims remain locked to repository artifacts, run IDs, and validator checks so manuscript updates do not drift from source data. The result is a decision-system framing in which calibration, cost, intervention behavior, and governance are evaluated together under faulted operating conditions.

Keywords: Machine Learning, Energy Forecasting, Battery Dispatch, Robust Optimization, Conformal Prediction, Stochastic Programming, MLOps, Grid Operations

1 Introduction

1.1 Motivation: autonomy + telemetry dependence

Prediction-only systems do not deliver operational action. Grid operators need feasible dispatch decisions under uncertainty while balancing cost and carbon objectives. GridPulse is designed as a closed decision loop:

$$\text{Forecast} \rightarrow \text{Optimize} \rightarrow \text{Dispatch} \rightarrow \text{Measure} \rightarrow \text{Monitor}$$

1.2 Failure mode: telemetry degradation → miscalibrated uncertainty → unsafe dispatch

The central problem is integrated decision quality rather than isolated forecast error. The system must jointly satisfy five requirements: accurate short-horizon forecasting, uncertainty quantification usable in optimization, physically feasible dispatch, measurable impact vs. baselines, and reproducible claim traceability across manuscript formats.

1.3 Problem statement + research questions (RQs)

1. Can one architecture maintain strong forecasting quality across DE and US datasets with different operating regimes?
2. Does uncertainty-aware dispatch provide measurable value relative to deterministic dispatch?
3. What is the magnitude of cost/carbon/peak impact under dataset-scoped latest evidence locking?
4. Can run-scoped governance prevent cross-document metric contradictions?

1.4 RQ → Hypothesis → Test → Result Map

Table 1: Tight RQ-to-Hypothesis-to-Test-to-Result Mapping Under Locked Evidence

RQ	Hypothesis	Test protocol	Locked result	Decision
RQ1	A shared architecture can maintain strong forecasting quality across DE/US regimes.	Train/evaluate DE and US with the same temporal framing (24-hour horizon, 168-hour lookback, 10-fold CV, 24-hour gap); compare RMSE/MAE/sMAPE/R^2 in dashboard metric artifacts.	GBM is the strongest family across locked DE/US targets; representative load RMSE values are 267.51 (DE) and 162.89 (US).	Supported (within locked DE/US scope).
RQ2	Uncertainty-aware stochastic dispatch provides measurable value relative to deterministic stochastic formulation.	Read locked run-summary rows from `reports/research_metrics_de.csv` and `reports/research_metrics_us.csv`; evaluate VSS/EVPI under pinned run IDs.	VSS is positive in both locked runs: DE 2,708.61 (run `20260217_165756`) and US 297,092.71 (run `20260217_182305`).	Supported for stochastic value; impact percentages are reported separately as deterministic B1 vs B2.
RQ3	Dataset-scoped latest locking yields measurable, reproducible region-level impact magnitudes.	Extract latest dataset-scoped rows from `reports/impact_summary.csv` (DE) and `reports/eia930/impact_summary.csv` (US) using manifest rounding rules.	DE: 7.11% cost, 0.30% carbon, 6.13% peak. US: 0.11% cost, 0.13% carbon, 0.00% peak.	Supported.
RQ4	Run-scoped governance prevents cross-document metric contradictions at release time.	Enforce manifest/claim/validator contract across manuscript outputs: `paper/metrics_manifest.json`, `paper/claim_matrix.csv`, and `scripts/validate_paper_claims.py`.	Publication claims are run-ID scoped and validator-gated; canonical values cannot change without synchronized source and claim updates.	Supported as an empirical process-control property.

1.5 Contributions (3–5 verifiable bullets)

1. A production-oriented multi-layer GridPulse system spanning data, forecasting, optimization, monitoring, and serving.
2. A conformal and adaptive interval workflow integrated with robust dispatch evaluation.
3. A dataset-scoped latest metric lock with explicit run IDs and source traceability.
4. A publication governance mechanism (manifest, claim matrix, validator) that blocks legacy metric regression.

Contribution boundaries are explicit: this thesis does not claim universal market realism across all settlement regimes or universal transferability across all operators. It claims reproducible DE/US decision outcomes under the locked artifacts and governance policy.

1.6 Scientific novelty beyond integration

Most prior work treats forecasting, uncertainty, dispatch, and reporting as loosely coupled stages. This thesis makes three method-level claims beyond engineering integration:

1. **RAC-Cert as a control-coupled conformal method:** interval width is adapted by telemetry reliability, drift state, and dispatch sensitivity, then enforced through shielded repair and certificate persistence, coupling uncertainty construction directly to control risk.
2. **Truth-vs-observed safety semantics:** safety is evaluated on SOC^{true} while control actions are chosen from degraded SOC^{obs}, exposing hidden violation risk under telemetry faults that is masked by observed-state-only evaluation.
3. **Governance as an experimental object:** claim validity is machine-checkable through manifest-locked sources, claim-level provenance, and validator gates, making reproducibility a testable release condition rather than a narrative claim.

These novelty claims are evaluated only within the locked DE/US evidence scope and are not asserted as universal guarantees across all grids.

1.7 Paper organization

This draft is structured so sections can be expanded or trimmed without breaking metric consistency:

1. Sections 2–4 define implementation and methods with code-level anchors.
2. Sections 5–8 define evidence policy, traceability, and claim governance.
3. Section 6 provides canonical quantitative results.
4. Sections 9–13 cover interpretation, validity, limitations, and roadmap.
5. Appendices provide replication commands, artifact inventory, and copy-ready writing guidance.

If length must be reduced, preserve Sections 6, 7, and 14 first because they carry the metric-locked publication claims.

2 Background and Related Work

2.1 Forecast uncertainty & conformal prediction

GridPulse follows an eight-layer decision architecture implemented across repository modules:

1. Data ingestion and feature generation.
2. Forecast model training and inference.
3. Uncertainty interval calibration.
4. Anomaly detection.
5. Dispatch optimization (deterministic and robust).
6. Impact and stochastic evaluation.
7. Monitoring and retraining decisions.
8. API and dashboard serving.

Table 2: Architecture Components and Primary Code Locations

Layer	Purpose	Primary paths
Data pipeline	Feature generation and split construction	`src/gridpulse/data_pipeline/`, `src/gridpulse/pipeline/run.py`
Forecasting	GBM/LSTM/TCN training and inference	`src/gridpulse/forecasting/`
Uncertainty	Conformal + adaptive interval logic	`src/gridpulse/forecasting/uncertainty/conformal.py`
Optimization	Deterministic and robust dispatch solvers	`src/gridpulse/optimizer/lp_dispatch.py`, `src/gridpulse/optimizer/robust_dispatch.py`
Monitoring	Drift tests and retraining decisions	`src/gridpulse/monitoring/`
Safety	BMS checks and watchdog controls	`src/gridpulse/safety/`
API serving	API endpoint routing	`services/api/main.py`, `services/api/routers/`
Frontend	Operator dashboard and region views	`frontend/`

2.2 CQR and heteroscedastic uncertainty

2.3 Online drift/change detection in streaming systems

FastAPI serving endpoints include:

- `/forecast`
- `/forecast/with-intervals`
- `/anomaly`
- `/optimize`
- `/monitor`
- `/monitor/model-info`
- `/dc3s/step`
- `/dc3s/audit/\{command\_id\}`
- `/health`, `/ready`, `/metrics`

Additional control endpoints include `/system/health`, `/system/heartbeat`, and `/control/dispatch`, gated by API-key scope and safety validation. Existing runtime endpoints remain unchanged; these interfaces are additive extensions for Section 1–3 implementation coverage.

2.4 Robust optimization & risk-aware control (min–max, CVaR)

This thesis uses the following artifact families:

1. Dataset profile and dashboard metrics: `data/dashboard/*.json`.
2. DE impact summary: `reports/impact_summary.csv`.
3. US impact summary: `reports/eia930/impact_summary.csv`.

4. Stochastic run metrics: `reports/research_metrics_de.csv` and `reports/research_metrics_us.csv`.

5. Robustness diagnostics: `reports/publication/tables/table6_robustness.csv`.

These artifact contracts are treated as publication interfaces. If a contract path or schema changes, both `paper/metrics_manifest.json` and `paper/claim_matrix.csv` must be updated in the same revision.

2.5 Networked control/IoT faults (dropout, delay, stale, out-of-order)

GridPulse supports three execution modes:

1. Offline batch mode for training, evaluation, and report generation.

2. API service mode (`services/api/main.py`) for live forecast and optimization requests.

3. Dashboard mode (`frontend/`) for operator-facing visual workflows.

The system is documented with safe degradation paths (for example, fallback baseline dispatch on infeasible optimization states) rather than assuming uninterrupted perfect runtime conditions.

2.6 Reproducible research artifacts and evaluation governance

An operational request path is:

1. Load latest feature window and resolve requested targets.

2. Retrieve model bundle and generate forecast vectors.

3. Attach conformal intervals when calibration artifacts exist.

4. Solve deterministic or robust dispatch with configured constraints.

5. Return action plan and cost/carbon summaries when available.

6. Emit monitoring signals and preserve control-plane heartbeat state.

2.7 Positioning summary: what's missing and why DC^{3}S

Section 1 (immediate publish blockers) was implemented by replacing frontend chat-tool mock payloads with backend-derived FastAPI calls in `frontend/src/app/api/chat/tool-executors.ts`. Production tool paths now resolve through `/forecast`, `/forecast/with-intervals`, `/optimize`, `/optimize/baseline`, `/monitor`, `/anomaly`, `/health`, `/ready`, and `/monitor/model-info`. Tool execution explicitly fails on backend errors instead of returning fabricated fallback payloads. Deployment/runtime hygiene updates were applied as well: dashboard ports were normalized to `3000`, dashboard deploy workflow Dockerfile targeting was corrected, local absolute report paths were removed, and coverage artifacts were removed/ignored for reproducibility hygiene.

Section 2 (streaming correctness) was implemented with a new CLI module `src/gridpulse/streaming/run_consumer.py` supporting: `python -m gridpulse.streaming.run_consumer -config configs/streaming.yaml -max-messages N`. This loader maps YAML config into streaming `AppConfig`, instantiates `StreamingIngestConsumer`, and closes resources cleanly. Persistence behavior was made explicit via public `write_event(...)` in `src/gridpulse/streaming/consumer.py` and integrated in `src/gridpulse/streaming/worker.py` after schema validation. The expected persistence artifact is `data/interim/streaming.duckdb` with table `telemetry_events`.

Section 3 (DC3S) was implemented with new config `configs/dc3s.yaml`, DC3S modules in `src/gridpulse/dc3s/` (`quality.py`, `drift.py`, `calibration.py`, `shield.py`, `certificate.py`,

state.py), and FastAPI router endpoints in services/api/routers/dc3s.py: POST/dc3s/step and GET/dc3s/audit/\{command_id\}. The step endpoint returns proposed and shield-repaired safe actions, uncertainty payloads, and certificate identifiers; audit retrieval returns persisted certificate JSON. The heuristic controller mode is intentionally not implemented in this phase and is explicitly rejected at runtime.

Frontend live-ops behavior for this flow is explicitly implemented in frontend/src/app/(dashboard)/page.tsx, frontend/src/lib/api/dc3s-client.ts, frontend/src/components/dashboard/DC3SLiveCard.tsx, and frontend/src/app/api/dc3s/audit/[commandId]/route.ts. The dashboard initializes DC3S polling at 15 seconds, supports operator-selected auto-refresh (Off, 5s, 10s, 15s, 30s, 60s), and provides manual refresh. The live card exposes a command-linked audit route (/api/dc3s/audit/\{commandId\}) that proxies to FastAPI /dc3s/audit/\{commandId\} with explicit error propagation.

3 Problem Formulation and Safety Definitions

3.1 Time index, horizon, and notation

We use discrete time $t \in \{0, \ldots, H-1\}$, horizon H, and $\Delta t = 1$ hour in canonical experiments. Core symbols are standardized to avoid notation drift across control and evaluation sections. This manuscript follows the dashboard profile lock in data/dashboard/manifest.json generated at 2026-02-17T11:15:38.623283.

Table 3: Core notation

Symbol	Meaning
SOC_t^{true}	True plant state used for safety evaluation.
SOC_t^{obs}	Telemetry-observed state used by controller.
$U_t = [L_t, U_t]$	Uncertainty set for robust/risk-aware dispatch at step t.
$\hat{q}$	CQR correction quantile from calibration split.
w_t	Telemetry reliability score in $[0, 1]$.
η, u_s	CVaR variables (VaR anchor and tail slacks).

Table 4: Canonical Dataset Profile Lock

Region	Rows (hourly)	Columns	Engineered Features	Approx Days
DE	17,377	98	94	724.04
US	13,638	118	114	568.25

Date ranges (UTC):

- DE: 2018-10-07 23:00 to 2020-09-30 23:00
- US: 2024-07-01 06:00 to 2026-01-20 11:00

3.2 Battery dynamics and constraints

Both regions use three target variables: load_mw, wind_mw, and solar_mw. The DE profile includes price/carbon channels specific to OPSD-based processing, while the US profile includes EIA-930 generation-mix channels and region-specific price/carbon features.

Table 5: Engineered Feature Family Counts

Region	Weather Features	Lag Features	Calendar Features	Total Features
DE	40	36	6	94
US	56	42	6	114

3.3 Grid interaction constraints (import cap, balance)

3.4 Uncertainty set definition U_t

We define the interval uncertainty set as:

$$U_t = [L_t, U_t], \qquad L_t = \hat{y}_t - \hat{q}_t, \quad U_t = \hat{y}_t + \hat{q}_t.$$

For RAC-Cert, $\hat{q}_t$ is adapted by reliability and sensitivity multipliers before robust/CVaR solving.

Table 6: Target Statistics from Dashboard Profile Files

Region	Target	Mean (MW)	Std (MW)	Min (MW)	Max (MW)	Median (MW)	Non-zero (%)
DE	`load_mw`	55,156.42	9,998.06	31,923.0	76,925.0	54,752.0	100.0
DE	`wind_mw`	14,368.98	10,321.58	136.0	46,064.0	11,728.0	100.0
DE	`solar_mw`	5,013.16	7,665.92	0.0	32,947.0	156.0	56.8
US	`load_mw`	75,757.21	11,819.46	52,940.0	120,343.0	73,564.0	100.0
US	`wind_mw`	10,955.94	6,041.83	0.0	26,132.0	10,183.5	100.0
US	`solar_mw`	2,875.42	3,842.56	0.0	14,315.0	110.0	94.6

3.5 Telemetry degradation model and belief vs truth states

Legacy profile claims from older report families are out of scope unless explicitly reconciled to this dashboard lock. This avoids reintroducing conflicting row/date/feature statements.

3.6 Safety metrics: violation rate + severity (P95)

Both DE and US training configs use a 24-hour forecast horizon and 168-hour lookback with time-aware cross-validation and an explicit 24-hour gap (`cross_validation.n_folds = 10`, `cross_validation.gap = 24`). This temporal framing reduces leakage risk and protects downstream decision-impact validity.

3.7 What is guaranteed vs empirical (assumptions)

Guaranteed properties are command-level feasibility after shielding/projection and certificate persistence integrity. Empirical properties include coverage, violation reduction, and transfer performance. Assumptions include synchronized timebase, consistent efficiency parameters, and valid truth-state simulation in CPSBench.

4 System Overview: GridPulse Execution Pipeline

4.1 Control loop and execution boundary

Model Families

GridPulse trains three forecasting families per target:

1. GBM (LightGBM), used as the primary operational model in the locked dashboard snapshots.

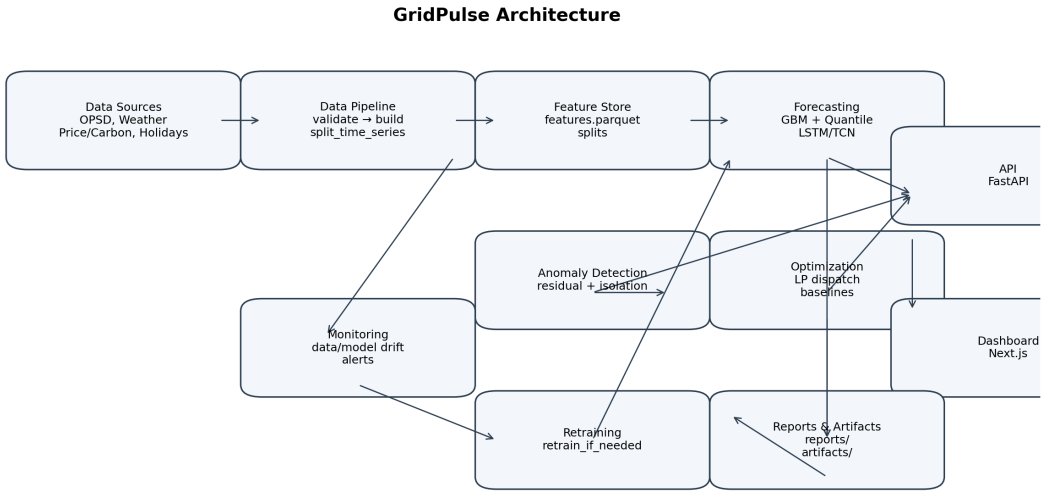

Figure 1: GridPulse execution pipeline and DC^{3}S placement at the execution boundary.

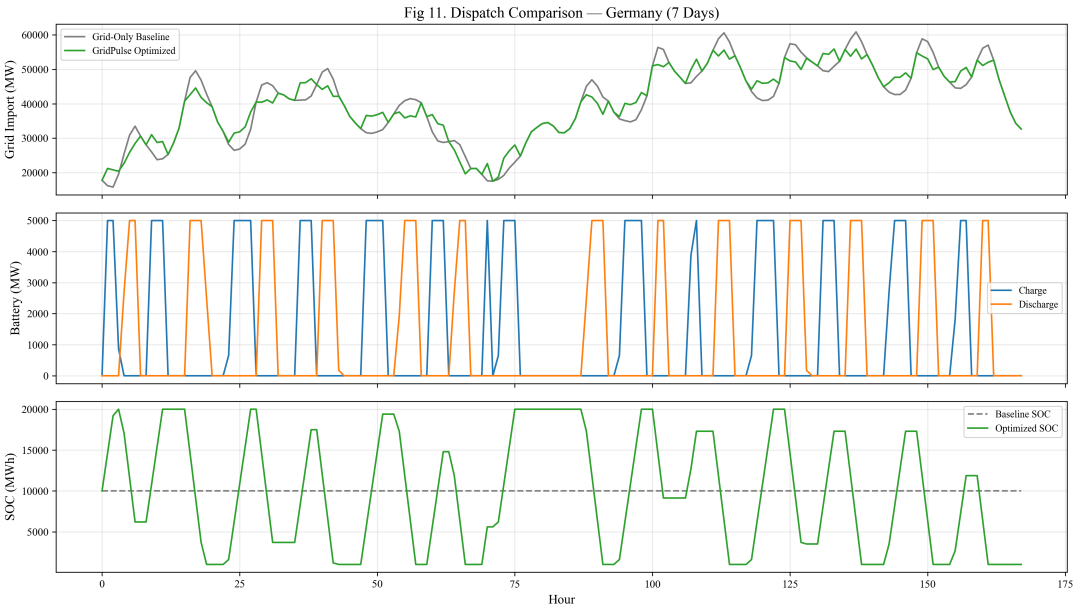

Figure 2: DC^{3}S runtime step flow: observe → score → detect → bound → solve → repair → certify.

2. LSTM sequence model.
3. TCN sequence model.

Training Setup

From `configs/train_forecast.yaml` and `configs/train_forecast_eia930.yaml`:

- Forecast horizon: 24 hours.
- Lookback context: 168 hours.
- Time-aware CV configured (10 folds, 24-hour gap).
- Seeded, reproducible training controls.

Implementation notes important for reproducibility:

1. Both regions keep GBM enabled as baseline operational model with explicit LightGBM parameter blocks.
2. LSTM and TCN blocks are present in both configs, with region-specific parameterization.
3. Cross-validation behavior is explicit in config rather than implicit defaults.
4. Training outputs are persisted into region-scoped artifact/report paths.

Runtime training controls are explicit in `src/gridpulse/forecasting/train.py` and `scripts/train_dataset.py`. `-tune` force-enables Optuna tuning, `-no-tune` disables tuning even when enabled in YAML, `-ensemble` trains multi-seed GBM members, `-max-seeds` caps ensemble size, and `-n-trials`/`-top-pct` override trial-count and top-trial aggregation behavior for tuned runs. Saved bundles persist selected tuning metadata and `ensemble_models`/seed-member diagnostics so inference and report generation remain run-reproducible.

Metric Interpretation

Primary comparison metrics are RMSE, MAE, sMAPE, and R^2. MAPE is recorded but can be numerically unstable in low-denominator regimes (notably solar nighttime and near-zero wind segments), so interpretation prioritizes sMAPE and absolute error metrics. Interpretation policy:

1. RMSE and MAE are primary for magnitude-sensitive operational quality.
2. sMAPE supports relative comparability across differently scaled targets.
3. R^2 is retained as fit context, not a standalone decision metric.
4. MAPE is treated as supplementary in near-zero target regimes.

Forecast Inference Contract

API-level forecast inference (`services/api/routers/forecast.py`) follows a strict contract:

1. Load features from configured path.
2. Resolve model bundle by explicit or fallback target mapping.
3. Emit next-horizon forecast outputs with metadata.
4. Surface missing-model states explicitly (not silent fallback fabrication).

Resolution behavior is deterministic: explicit per-target model paths are checked first, then ordered fallback search (`lstm`, `tcn`, `gbm`) is applied under `artifacts/models`. If `ensemble_models` is present in a GBM bundle, inference averages member predictions; quantiles are emitted from explicit quantile models when present, otherwise from stored residual quantile offsets.

4.2 Component interfaces (inputs/outputs)

Conformal calibration at $\alpha = 0.10$ is implemented in the uncertainty module (`conformal.py`). For point forecast $\hat{y}$ and calibrated residual quantile q, interval bounds are:

$$\text{Lower} = \hat{y} - q, \qquad \text{Upper} = \hat{y} + q.$$

The implementation supports horizon-wise quantiles, rolling updates, and adaptive scaling behavior for online coverage control. It combines two mechanisms: `ConformalInterval` for calibrated interval construction and `AdaptiveConformal` for bounded online alpha updates under miss events. Default code-level settings include `alpha = 0.10`, `rolling = true`, and `rolling_window = 720`.

4.3 Runtime modes (simulation, shadow, active)

The deterministic optimizer in `src/gridpulse/optimizer/lp_dispatch.py` solves a mixed-integer linear formulation with:

- battery charge/discharge decisions,
- grid import,
- SOC dynamics and constraints,
- cost and carbon objective terms,
- degradation and peak penalties.

Representative objective form:

$$\min \sum_t \left(c_t^{\text{energy}} G_t + c_t^{\text{carbon}} G_t + c^{\text{deg}} (P_t^{\text{ch}} + P_t^{\text{dis}}) \right).$$

Key implementation behaviors:

1. Piecewise battery efficiency regimes (`efficiency_regime_a`, `efficiency_regime_b`) with SOC split gating.
2. Optional interval-aware risk binding via `risk.mode = worst_case_interval`.
3. High unmet-load and curtailment penalties to avoid unsafe economic shortcuts.

Locked default config highlights in `configs/optimization.yaml`: `objective.cost_weight = 1.0`, `objective.carbon_weight = 1.2`, battery capacity 20,000 MWh, battery max power 5,000 MW, grid import cap 100,000 MW.

4.4 Auditing and certificate records

The robust optimizer in `src/gridpulse/optimizer/robust_dispatch.py` solves a two-scenario min-max linear program over lower/upper load bounds. With epigraph variable z and scenario costs C_s:

$$\min \lambda z + (1 - \lambda)\bar{C} + c^{\text{deg}} \,\text{throughput}, \qquad z \geq C_s \ \forall s.$$

The solver returns feasibility flags, scenario costs, and dispatch trajectories for analysis. Additional behavior:

1. Charge/discharge decisions are shared across scenarios; grid import and SOC remain scenario-indexed.
2. Non-optimal termination returns explicit infeasible-safe payloads rather than partial unsafe commands.
3. Solver availability and status are explicit in outputs used for publication metrics.

4.5 Artifact-locked reproducibility workflow

DC3S is implemented in `src/gridpulse/dc3s/` and exposed through `POST/dc3s/step` and `GET/dc3s/audit/\{command\_id\}` in `services/api/routers/dc3s.py`. It composes telemetry quality weighting, drift detection, uncertainty inflation, safe action repair, and certificate-grade audit persistence.

The implemented interval inflation law is linear and bounded:

$$\text{infl} = \text{clip}\left(1 + k_q(1 - w_t) + k_d \mathbf{1}[\text{drift}],\ 1,\ \text{infl}_{\max}\right)$$

$$\text{Lower} = \hat{y} - q \cdot \text{infl}, \qquad \text{Upper} = \hat{y} + q \cdot \text{infl}$$

$$w_t \leftarrow \max(w_t, \min_w)$$

The reliability module (`quality.py`) computes w_t from missingness, delay, out-of-order behavior, and spike penalties. Drift logic (`drift.py`) uses Page-Hinkley updates on residual magnitude $r_t = |y_t - \hat{y}_t|$ with configured warmup and cooldown controls.

Safety shielding (`shield.py`) supports two modes:

1. `projection`: deterministic clipping against SOC, power, and ramp constraints.
2. `robust_resolve`: attempt robust optimization under interval bounds, then project as final guard.

Certification (`certificate.py`) computes `model_hash` and `config_hash`, supports optional `prev_hash` chaining, and stores signed payload structure to DuckDB. Runtime online state (`state.py`) persists per `(zone, device, target)` keys including drift state, last inflation, latest telemetry/action context, and audit linkage pointers.

4.6 Monitoring and Retraining Controls

Monitoring logic combines data-drift and model-drift checks:

- KS-based feature distribution drift in `src/gridpulse/monitoring/data_drift.py`.
- Model degradation thresholds with retraining decisions in the monitoring retraining module.
- API monitoring endpoints in `services/api/routers/monitor.py`.

Configured default thresholds in the locked snapshot:

1. `data_drift.p_value_threshold = 0.01`
2. `model_drift.metric = mape`
3. `model_drift.degradation_threshold = 0.15`
4. `retraining.cadence_days = 30`
5. `retraining.min_new_data_days = 14`

4.7 Safety and Operational Controls

Safety mechanisms include:

1. BMS validation for dispatch commands.
2. Watchdog heartbeats and island-mode protection.
3. API key scope checks for read/write control routes.
4. Health and readiness endpoints for deployment gating.

The `/control/dispatch` route is protected by three gates: write-scope authorization, watchdog islanding check, and BMS constraint validation on SOC/power commands.

4.8 Reproducibility and Governance Controls

Publication claims are locked via:

- `paper/metrics_manifest.json`
- `paper/claim_matrix.csv`
- `scripts/validate_paper_claims.py`

These are release gates, not passive documentation: a manuscript change is incomplete unless validator checks pass and claim provenance remains traceable.

4.9 Method Assumptions and Boundary Conditions

This thesis assumes hourly timestep consistency, proxy suitability of price/carbon channels for comparative decision analysis in locked windows, interval-based uncertainty representation, and configured battery/grid constraints sufficient for physically plausible dispatch plans. These are engineering assumptions, not universal truths.

4.10 Baselines and Comparator Policies

This manuscript uses three explicit baseline policies for decision comparisons:

1. **B1 (Deterministic LP, no uncertainty intervals):** implemented in `src/gridpulse/optimizer/lp_dispatch.py` via `optimize_dispatch(...)` without `load_interval` / `renewables_interval`. It uses battery + grid constraints and is the optimization policy reported as GridPulse in canonical impact artifacts.
2. **B2 (Grid-only, no battery):** implemented in `src/gridpulse/optimizer/baselines.py` as `grid_only_dispatch(...)`. It sets `battery_charge_mw=0` and `battery_discharge_mw=0` and serves net deficit from grid import. This is the denominator baseline for canonical DE/US impact percentages.
3. **B3 (Naive battery heuristic):** implemented in `src/gridpulse/optimizer/baselines.py` as `naive_battery_dispatch(...)` with a fixed rule that charges at 00–05 and discharges at 17–21.

Runtime disambiguation: while API `/optimize` may default to robust mode in service context, publication impact percentages in this thesis are produced from offline deterministic `optimize_dispatch(...)` (B1) versus grid-only baseline (B2).

4.11 RAC-Cert Unified Uncertainty-Control Method (Flagship)

To convert interval construction from a fixed post-processing layer into a control-aware method class, this manuscript defines **RAC-Cert** (Reliability-Adaptive Conformal with Certified Robustness). RAC-Cert is implemented in `src/gridpulse/dc3s/rac_cert.py` and consumed by calibration/runtime paths in `src/gridpulse/dc3s/calibration.py` and `src/gridpulse/cpsbench_iot/runner.py`.

Notation and Calibration Objects

At each time step t, let $\hat{y}_t$ denote point forecast, q_t^{lo} and q_t^{hi} denote base quantile forecasts, $w_t \in [0, 1]$ denote telemetry reliability, and s_t denote dispatch sensitivity. On calibration index i, nonconformity is:

$$\text{score}_i = \max\left(q_i^{lo} - y_i,\ y_i - q_i^{hi},\ 0\right).$$

Calibration points are assigned to volatility bin k using rolling volatility features (default three bins: low/mid/high). For each bin:

$$\hat{q}_k = \text{Quantile}_{1-\alpha}\left(\text{score}_i \mid i \in k\right), \qquad \hat{q}_{global} = \text{Quantile}_{1-\alpha}\left(\text{score}_i\right).$$

To stabilize sparse bins, RAC-Cert applies shrinkage:

$$\hat{q}_k^{shrunk} = \frac{n_k}{n_k + \tau}\hat{q}_k + \frac{\tau}{n_k + \tau}\hat{q}_{global},$$

where τ is configurable (`qhat_shrink_tau`).

Reliability + Sensitivity Adaptive Width Law

At runtime, RAC-Cert computes an effective half-width multiplier:

$$m_t = 1 + \beta_r(1 - w_t) + \beta_s \cdot \tilde{s}_t,$$

with normalized sensitivity

$$\tilde{s}_t = \text{clip}\left(\frac{s_t}{s_{ref}}, 0, 1\right).$$

Effective conformal radius is:

$$q_t^{eff} = \min(\hat{q}_{k_t}^{shrunk} \cdot m_t,\ q_{max} \cdot \hat{q}_{k_t}^{shrunk}),$$

where q_{max} is `max_q_multiplier`. Drift/reliability inflation remains explicit:

$$\text{infl}_t = \text{clip}\left(1 + k_q(1 - w_t) + k_d \mathbf{1}[\text{drift}] + k_s \tilde{s}_t,\ 1,\ \text{infl}_{max}\right).$$

Base RAC interval:

$$L_t = \hat{y}_t - q_t^{eff} \cdot \text{infl}_t, \qquad U_t = \hat{y}_t + q_t^{eff} \cdot \text{infl}_t.$$

Then ambiguity widening from `widen_bounds(...)` is applied as a final robustification stage.

Coverage statement (informal). The DC3S inflation rule is intended to tie per-step miscoverage to the effective reliability-adjusted level $\delta \cdot w_t$, so low-reliability telemetry widens intervals rather than silently preserving nominal width. Informally, this means the method targets stricter uncertainty inflation when telemetry quality falls, while still relying on adaptive conformal logic for running-average coverage control rather than claiming a pointwise theorem for every step. This should be read as a method sketch, not a formal proof: the relevant background is adaptive/running-average conformal inference [Citation placeholder: Gibbs & Candès, JMLR 2024], weighted conformal ideas for non-uniform validity [Citation placeholder: Barber, Candès, Ramdas, Tibshirani, Annals of Statistics 2023], and the subgaussian tail-bound intuition used in the inflation heuristic [Citation placeholder: Vershynin, 2018].

Controller-Consistent RAC Runtime

RAC-Cert is applied to all controllers in CPSBench runner while preserving controller names for comparability:

1. `deterministic_lp`: RAC intervals with neutral reliability defaults.
2. `robust_fixed_interval`: RAC intervals used by robust solver.
3. `cvar_interval`: RAC intervals used to sample CVaR scenarios.
4. `dc3s_wrapped`: full reliability + drift + sensitivity adaptation with certificate logging.

This design avoids mixed uncertainty semantics across controllers and keeps ablation interpretation clean.

Sensitivity Signal

RAC-Cert supports two probe modes:

1. **Heuristic probe** (release default): deterministic low-cost estimator for throughput.
2. **Finite-difference probe** (diagnostic): perturb load by $\pm\epsilon$ and measure first-step net battery-action response.

Sensitivity metadata is persisted as: `sensitivity\_t`, `sensitivity\_norm`, `q\_eff`, `q\_multiplier`, and `inflation\_components`.

Certificate and Main-Table Contract

Publication outputs must include RAC diagnostics in `reports/publication/dc3s_main_table.csv`:

- `rac\_sensitivity\_mean`
- `rac\_sensitivity\_p95`
- `rac\_q\_multiplier\_mean`
- `rac\_q\_multiplier\_p95`
- `rac\_inflation\_mean`

DC3S certificates additionally include: `coverage\_lb\_t`, `sensitivity\_t`, `sensitivity\_norm`, `q\_eff`, `q\_multiplier`, `interval\_width`, and `guarantee\_checks\_passed`.

Truth-vs-Observed SOC Safety Semantics

Safety is evaluated on physical truth state (`soc\_true`), while control decisions are based on degraded telemetry (`soc\_obs`) under dropout/stale/noise faults. This closes the common evaluation gap where controller-visible state is conflated with plant truth.

$$\text{violation}_t = \mathbf{1}\{SOC_t^{true} < SOC_{min} \ \vee \ SOC_t^{true} > SOC_{max}\}.$$

Severity is measured as boundary-distance MWh violation, including P95 severity for release-gate reporting.

5 Forecasting and Regime-Aware CQR Uncertainty

5.1 Forecasting target(s), features, and training objective

All publication-facing values in this thesis must map to canonical entries in `paper/metrics_manifest.json`. The markdown master file is `paper/PAPER_DRAFT.md`.

5.2 Quantile forecasting (q10/q50/q90)

5.3 CQR calibration split and correction $\hat{q}$

Conformal correction is computed from calibration nonconformity:

$$s_i = \max(q^{lo}(x_i) - y_i,\ y_i - q^{hi}(x_i),\ 0), \qquad \hat{q} = \text{Quantile}_{1-\alpha}(s_i).$$

Corrected interval:

$$\left[q^{lo}(x) - \hat{q},\ q^{hi}(x) + \hat{q}\right].$$

Table 7: Canonical Source-of-Truth Mapping

Claim Domain	Source Artifact	Selection Rule
DE decision impact	`reports/impact_summary.csv`	latest dataset-scoped record
US decision impact	`reports/eia930/impact_summary.csv`	latest dataset-scoped record
DE stochastic metrics	`reports/research_metrics_de.csv`	`run_id=20260217_165756`, `row_type=run_summary`
US stochastic metrics	`reports/research_metrics_us.csv`	`run_id=20260217_182305`, `row_type=run_summary`
Dataset profile claims	`data/dashboard/*_stats.json` + manifest	dashboard profile lock `2026-02-17T11:15:38.623283`

1. Percent values: `round(value*100,2)`.
2. Monetary/mass/power values: two-decimal display where reported.
3. Large stochastic values: thousands separators plus two decimals.

5.4 Regime binning (volatility bins) and group coverage

Table 8: Regime-group CQR coverage and interval width summary.

target	group	bin	picp_90	mean_width	sample_count
load_mw	low	0	0.867446	1400.418976	513
load_mw	med	1	0.816092	1247.121245	261
load_mw	high	2	0.887691	836.839593	1763

1. Compare methods and process governance across regions.
2. Do not enforce equal effect-size expectations across regions.
3. Keep all stochastic claims tied to region-specific run IDs.

5.5 Uncertainty metrics (PICP@90, width, sharpness–calibration tradeoff)

The manuscript is release-ready only when:

1. `pythonscripts/validate_paper_claims.py` passes.
2. Required run IDs and canonical values are present in markdown and LaTeX.
3. No banned legacy patterns or placeholders appear.
4. Publication claims map to `paper/claim_matrix.csv`.

5.6 Reconciliation Procedure for Conflicts

When a conflicting claim is found:

1. Locate claim text and claim ID in `paper/claim_matrix.csv`.
2. Identify canonical source and extraction rule in `paper/metrics_manifest.json`.

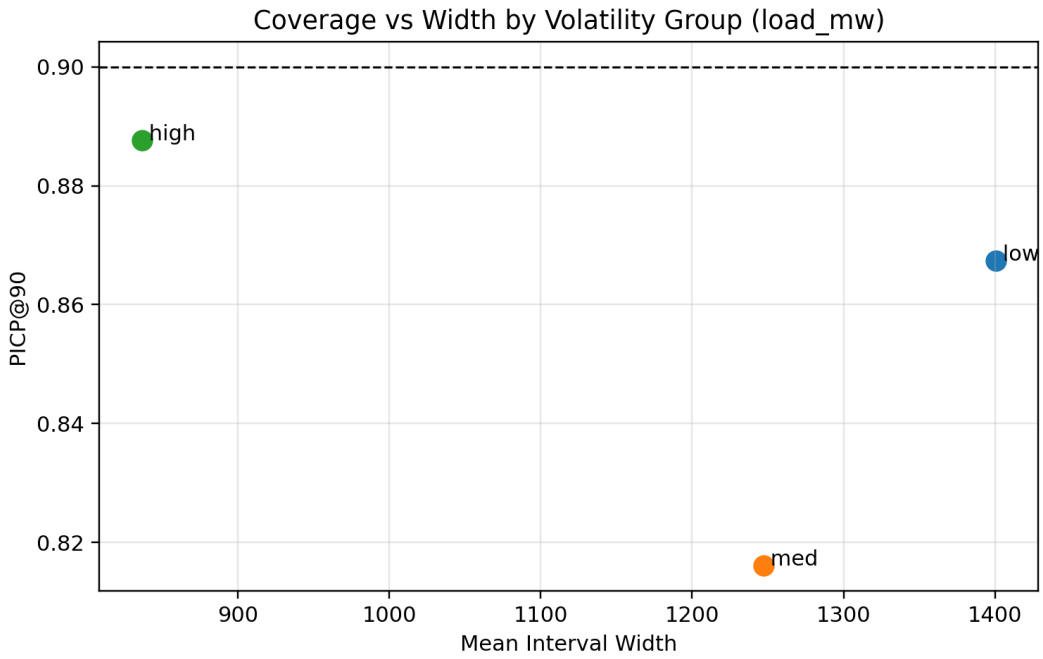

Figure 3: Regime-aware CQR coverage and width behavior by volatility bin.

3. Replace manuscript value using the locked rounding policy.
4. Re-run `pythonscripts/validate_paper_claims.py`.
5. Mirror correction in markdown, LaTeX, and DOCX sync outputs.

This blocks partial fixes where one format is updated and another remains stale.

6 Telemetry Reliability Scoring and Drift Detection

6.1 Telemetry fault signals and reliability score w_t

Reliability is modeled as a bounded aggregate of telemetry degradation signals:

$$w_t = \text{clip}(1 - \lambda_d d_t - \lambda_s s_t - \lambda_o o_t - \lambda_p p_t,\ 0,\ 1)\,,$$

where d_t is dropout, s_t stale telemetry, o_t out-of-order behavior, and p_t spike/noise penalty. Source: `data/dashboard/de_metrics.json`, `data/dashboard/us_metrics.json`.

Germany (DE)

United States (US)

GBM is the strongest model family in the locked operational snapshots across both regions and all targets. Thesis-level interpretation:

1. GBM leads across load, wind, and solar in both regions for the locked artifacts.
2. Deep models remain informative but are not operational leaders in this snapshot.
3. Forecast rankings should be read together with downstream optimization outcomes.

Table 9: Forecast Metrics by Model and Target (DE)

Target	Model	RMSE	MAE	sMAPE (%)	R^2
`load_mw`	GBM	267.51	167.49	0.35	0.9991
`load_mw`	LSTM	3,474.78	2,722.50	5.38	0.8498
`load_mw`	TCN	2,668.05	2,031.12	3.94	0.9114
`wind_mw`	GBM	183.93	118.19	2.29	0.9993
`wind_mw`	LSTM	6,735.07	5,511.23	60.85	0.1545
`wind_mw`	TCN	9,196.16	7,167.41	76.67	-0.5763
`solar_mw`	GBM	251.44	121.22	69.27	0.9992
`solar_mw`	LSTM	4,079.38	2,835.74	109.44	0.8007
`solar_mw`	TCN	2,702.34	1,583.05	93.76	0.9125

Table 10: Forecast Metrics by Model and Target (US)

Target	Model	RMSE	MAE	sMAPE (%)	R^2
`load_mw`	GBM	162.89	123.23	0.17	0.9996
`load_mw`	LSTM	4,767.14	3,835.32	5.24	0.6014
`load_mw`	TCN	3,850.49	2,877.31	3.82	0.7399
`wind_mw`	GBM	269.23	144.66	1.75	0.9982
`wind_mw`	LSTM	6,301.91	5,234.31	42.59	0.0627
`wind_mw`	TCN	7,187.54	5,930.74	47.65	-0.2192
`solar_mw`	GBM	208.92	74.93	45.45	0.9962
`solar_mw`	LSTM	1,781.97	1,055.34	136.92	0.7135
`solar_mw`	TCN	1,743.66	965.08	131.50	0.7257

Table 11: Residual Quantiles and Coverage by Region/Target

Region	Target	Residual q10	Residual q50	Residual q90	PICP (%)
DE	`load_mw`	-321.56	-11.61	254.52	95.17
DE	`wind_mw`	-293.49	-42.68	221.95	88.15
DE	`solar_mw`	-165.27	0.40	336.66	87.00
US	`load_mw`	-240.08	1.00	223.77	87.39
US	`wind_mw`	-146.14	-3.96	122.64	80.50
US	`solar_mw`	-105.40	-0.14	186.94	90.47

6.2 Drift detection (residual-based test, e.g., Page–Hinkley)

Coverage behavior is near nominal for selected targets and lower for others, indicating region-target calibration opportunities. Coverage pattern notes:

1. DE load coverage is conservative relative to 90% nominal.
2. US wind coverage is materially below nominal and indicates under-coverage risk.
3. Solar coverage differs across regions, reflecting profile and feature differences.

6.3 Reliability-weighted ambiguity adaptation

Table 12: Baseline Mapping by Results Table

Result section/table	Numerator policy	Denominator/comparator policy	Source artifact
Decision Impact (Canonical Percent Outcomes)	B1 deterministic LP (no uncertainty intervals)	B2 grid-only (no battery)	`reports/impact_summary.csv`; `reports/eia930/impact_summary.csv`
Decision Impact (Absolute Values)	B1 deterministic LP (no uncertainty intervals)	B2 grid-only (no battery)	`reports/impact_summary.csv`; `reports/eia930/impact_summary.csv`
Stochastic Value Metrics (Canonical Run IDs)	robust vs deterministic stochastic formulations	deterministic/robust run-family comparator (not B2 denominator)	`reports/research_metrics_de.csv`; `reports/research_metrics_us.csv`

6.4 Fail-safe logic when telemetry is poor (hold/constraint tightening)

Source files: `reports/impact_comparison.json` (DE), `reports/eia930/impact_comparison.json` (US). The table uses existing serialized artifacts only (`baseline` = B2, `naive` = B3, `optimized_forecast` = B1) with two-decimal display rounding.

Table 13: Baseline Absolute Outcomes by Region and Policy

Region	Policy ID	Policy name	Cost (USD)	Carbon (kg)	Peak (MW)
DE	B2	Grid-only (no battery)	154,773,462.72	1,461,641,243.28	52,165.00
DE	B3	Naive battery heuristic	154,200,945.64	1,454,657,899.03	52,165.00
DE	B1	Deterministic LP (no uncertainty intervals)	143,776,429.88	1,457,205,025.31	48,969.37
US	B2	Grid-only (no battery)	461,364,890.00	3,907,803,618.32	74,741.00
US	B3	Naive battery heuristic	462,821,035.03	3,911,194,390.28	70,929.00
US	B1	Deterministic LP (no uncertainty intervals)	460,842,677.76	3,902,642,369.72	74,741.00

6.5 Decision Impact (Canonical Percent Outcomes)

Table 14: Canonical Decision Impact Percentages

Region	Cost Savings	Carbon Reduction	Peak Shaving
DE	7.11%	0.30%	6.13%
US	0.11%	0.13%	0.00%

Source files: `reports/impact_summary.csv`, `reports/eia930/impact_summary.csv`. All reported DE/US impact percentages are computed as B1 vs B2. Reading guidance:

1. Percent values are canonical publication claims.
2. Absolute outcomes provide scale interpretation.
3. DE and US should not be treated as directly interchangeable market contexts.

6.6 Decision Impact (Absolute Values)

Absolute outcomes show that modest percentages in large systems can still correspond to substantial absolute deltas.

Table 15: Absolute Cost and Carbon Outcomes

Region	Baseline Cost (USD)	GridPulse Cost (USD)	Cost Delta (USD)	Baseline Carbon (kg)	GridPulse Carbon (kg)	Carbon Delta (kg)
DE	154,773,462.72	143,776,429.88	10,997,032.84	1,461,641,243.28	1,457,205,025.31	4,436,217.97
US	461,364,890.00	460,842,677.76	522,212.24	3,907,803,618.32	3,902,642,369.72	5,161,248.60

Table 16: Absolute Peak Outcomes

Region	Baseline Peak (MW)	GridPulse Peak (MW)	Peak Delta (MW)
DE	52,165.00	48,969.37	3,195.63
US	74,741.00	74,741.00	0.00

6.7 Stochastic Value Metrics (Canonical Run IDs)

Table 17: Canonical Stochastic Metrics by Locked Run

Region	Run ID	Timestamp (UTC)	$\text{EVPI}_{\text{robust}}$	$\text{EVPI}_{\text{deterministic}}$	VSS	Feasible	Solver
DE	`20260217_165756`	2026-02-17T16:57:56.736552	2.32	-30.40	2,708.61	True	summary
US	`20260217_182305`	2026-02-17T18:23:06.287827	10,279,851.74	24,915,503.93	297,092.71	True	summary

US run-summary operational controls in the same canonical row include grid cap 69,844.27 MW, reserve SOC 2,000.00 MWh, terminal SOC target 4,450.43 MWh, load stress additive 1,257.02 MW, and stress interval multiplier 1.90. Interpretation:

1. Both canonical runs are feasible and explicitly tied to run IDs and timestamps.
2. DE run has modest EVPI magnitudes with positive VSS.
3. US run has large EVPI magnitudes with positive VSS, indicating strong uncertainty sensitivity.
4. These statements are run-scoped and should always include run IDs in text.

6.8 Robustness Perturbation Summary

Source: `reports/publication/tables/table6_robustness.csv`.

All listed perturbation points remain feasible in this summary table. Regret sign caution:

1. Negative regret values can occur depending on comparator definition and realization.
2. Regret interpretation must use consistent source-table definitions.

6.9 Cross-Region Interpretation

1. Both DE and US show positive VSS under canonical locked runs.
2. Magnitude differences are substantial, indicating sensitivity to regime and scenario design.
3. DE shows stronger direct percent impact in this snapshot.
4. US shows smaller direct percent impact but large stochastic value magnitudes.

This supports a core thesis claim: forecast-to-dispatch systems should be evaluated as region-specific decision systems, not only by aggregate forecast-score transferability.

Table 18: Additional Run-Summary Context

Region	Mean Dynamic Width	Mean Base Width	Mean Stressed Width	FACI Scale Lower	FACI Scale Upper
DE	653.57	657.11	not reported in DE schema	0.0648	1.0000
US	649.20	415.60	1,233.49	0.1035	1.5965

Table 19: Perturbation Robustness Summary

Dataset	Perturbation (%)	Infeasible (%)	Mean Regret
DE	0.0	0.0	0.00
DE	5.0	0.0	-1,509.47
DE	10.0	0.0	-68,063.96
DE	20.0	0.0	-183,810.02
DE	30.0	0.0	-142,933.97
US	0.0	0.0	0.00
US	5.0	0.0	25,431.64
US	10.0	0.0	-241,341.60
US	20.0	0.0	-140,118.36
US	30.0	0.0	506,266.38

6.10 Copy-Ready Core Claims

Use these exact strings to avoid drift:

1. DE cost savings: 7.11%; DE carbon reduction: 0.30%; DE peak shaving: 6.13%.
2. US cost savings: 0.11%; US carbon reduction: 0.13%; US peak shaving: 0.00%.
3. DE stochastic run: 20260217 _165756 ($\text{EVPI}_{\text{robust}}$ 2.32; $\text{EVPI}_{\text{deterministic}}$ -30.40; VSS 2,708.61).
4. US stochastic run: 20260217 _182305 ($\text{EVPI}_{\text{robust}}$ 10,279,851.74; $\text{EVPI}_{\text{deterministic}}$ 24,915,503.93; VSS 297,092.71).
5. Dataset-scoped latest policy is used; this manuscript is not a single common-run freeze.

6.11 Negative and Neutral Findings (Do Not Omit)

1. US peak shaving in locked impact sources is 0.00%.
2. Several interval coverage values fall below 90% nominal target.
3. Deep models are not top-ranked in locked dashboard snapshots.
4. Cross-region impact magnitude differs significantly despite shared architecture.

7 Dispatch Optimization: Robust and Risk-Aware Formulations

7.1 Deterministic dispatch formulation (baseline)

7.2 Robust min–max dispatch with interval uncertainty

A manuscript metric can be changed only if all conditions hold:

1. Value is updated in `paper/metrics_manifest.json` with source path and run ID.
2. Claim row exists or is updated in `paper/claim_matrix.csv`.

Table 20: Locked Source-of-Truth Artifacts

Claim Group	Canonical Source
DE impact metrics	`reports/impact_summary.csv`
US impact metrics	`reports/eia930/impact_summary.csv`
DE stochastic metrics	`reports/research_metrics_de.csv`, `run_id = 20260217_165756`
US stochastic metrics	`reports/research_metrics_us.csv`, `run_id = 20260217_182305`
Dataset profile claims	`data/dashboard/de_stats.json`, `data/dashboard/us_stats.json`, `data/dashboard/manifest.json`

3. `pythonscripts/validate_paper_claims.py` passes after edit.

4. Markdown and LaTeX manuscripts retain matching core metric statements.

This applies to percentages, absolute values, run IDs, timestamps, and any sentence embedding quantitative evidence.

7.3 Scenario construction from intervals/quantiles

Legacy report exports and older markdown writeups are treated as secondary references unless explicitly reconciled to this lock.

7.4 CVaR formulation as risk-aware baseline

CVaR is expressed via Rockafellar–Uryasev linearization:

$$\min \ \eta + \frac{1}{(1-\beta)S} \sum_{s=1}^{S} u_s, \qquad u_s \geq C_s - \eta, \quad u_s \geq 0.$$

Before approving a numeric edit:

1. Confirm source artifact path exists and is versioned.

2. Confirm row and field selection rule is unambiguous.

3. Regenerate display values from raw values using manifest rounding rules.

4. Verify metric synchronization in abstract, results, and conclusion.

5. Re-run validator and archive pass output in revision notes.

7.5 Computational complexity and solver configuration

Deterministic LP has the lowest runtime footprint; robust and CVaR variants increase variable/constraint counts through scenario dimensions and interval coupling. Canonical publication runs pin solver family, scenario counts, and seed list; staged short-horizon runs are used for correctness checks before full artifact regeneration.

8 DC^{3}S Safety Shield

8.1 Algorithm 1: DC^{3}S step (observe → score → detect → bound → solve → repair → certify)

1. Observe telemetry and compute reliability w_t.
2. Score residual behavior and drift indicators.
3. Build RAC-aware uncertainty bounds.
4. Solve deterministic/robust/CVaR dispatch candidate.
5. Repair/project command into safe feasible action space.
6. Persist certificate and audit metadata for traceability.

- **Verified**: directly traceable to locked source artifacts.
- **Conflicting**: contradicts canonical policy or locked values.
- **Unsupported**: no in-repo evidence found.
- **Needs Citation**: plausible but requires external source citation.

Claim statuses are operational controls; status changes require evidence/citation change, not wording-only edits.

8.2 Action repair/projection and constraint enforcement

Each claim entry in `paper/claim_matrix.csv` should include:

1. `claim_id`
2. `status`
3. `category`
4. `claim_text`
5. `source_file`
6. `source_locator`
7. `run_id` (when applicable)
8. `timestamp_utc` (when applicable)
9. `rounding_rule`

8.3 Guarantee checks (command-level safety)

Publication-facing sections should contain only:

1. Verified in-repo claims.
2. External claims that are explicitly cited.

8.4 Certificates and audit schema

1. Edit `paper/PAPER_DRAFT.md` as the master.
2. Run the claim validator.
3. Synchronize `paper/paper.tex`.
4. Synchronize DOCX last and record any environment/tooling limitations.

8.5 Practical considerations (latency budget, fallback thresholds)

1. Reviewer flags text spans or claim IDs.
2. Author resolves each item against `paper/claim_matrix.csv` and manifest sources.
3. Any unresolved numeric conflict blocks release.
4. Validator pass output is attached to the revision package.

9 Benchmark and Experimental Protocol

9.1 Datasets and regions (DE vs US)

Table 21: Dataset summary for publication protocol and benchmark scope.

Dataset	Country	Start	End	Rows	Signal	Non-Null	Coverage%
Germany (OPSD)	DE	2018-10-07	2020-09-30	17377	load_mw	17377	100.000000
Germany (OPSD)	DE	2018-10-07	2020-09-30	17377	wind_mw	17377	100.000000
Germany (OPSD)	DE	2018-10-07	2020-09-30	17377	solar_mw	17377	100.000000
USA (EIA-930)	US	2015-07-08	2026-01-20	92382	load_mw	92382	100.000000
USA (EIA-930)	US	2015-07-08	2026-01-20	92382	wind_mw	92382	100.000000
USA (EIA-930)	US	2015-07-08	2026-01-20	92382	solar_mw	92382	100.000000

1. DE and US impact percentages are directly reproducible from canonical impact CSVs.
2. DE and US stochastic values are directly reproducible from canonical run-summary rows.
3. GBM is the top forecast performer in locked dashboard snapshots.
4. Governance controls materially reduce manuscript drift risk.

The strongest thesis claim is the combined result of measurable decision outcomes and explicit provenance enforcement.

9.2 CPSBench truth vs observed SOC design

Likely contributors include differences in data window, operating context, feature-space composition, and stress/scenario settings. This thesis reports region-scoped outcomes and does not merge them into a single universal effect claim. Deployment implication: a strategy with high percentage gains in one region can remain valuable in another while producing smaller direct percentages.

9.3 Fault suite and severity sweeps

The thesis does not claim universal superiority across all grids. All quantitative statements are tied to the locked DE/US evidence scope.

9.4 Baselines and dc3s_wrapped controller

The primary engineering contribution is integrated, auditable decision evidence: forecasting, uncertainty, optimization, and governance are connected in one reproducible workflow.

9.5 Statistical protocol (multi-seed, CI, paired tests)

Operationally, GridPulse is a guarded decision-support stack:

1. Healthy forecast and interval artifacts trigger optimization-driven dispatch plans.
2. Infeasible or degraded states trigger explicit safe fallbacks.
3. Watchdog and BMS controls enforce safety before dispatch acceptance.

Operationally, the stack is ready for software deployment pathways and controlled integration tests, but hardware field validation remains pending in the current evidence lock.

9.6 Transfer cases (DE→US, US→DE, seasonal shift ×2)

Transfer evaluation explicitly includes four cases: `DE_to_US_no_retrain`, `US_to_DE_no_retrain`, `DE_season_shift`, and `US_season_shift`, each tested under nominal and telemetry-fault sweeps.

10 Results

10.1 Uncertainty calibration + group coverage

Risk: metric drift from mixed artifacts. Mitigation: manifest lock, claim matrix, and automated validator checks.

10.2 Safety under telemetry faults (violation rate + severity)

Table 22: Main safety and cost outcomes under telemetry faults.

controller	true_soc_violation_rate_mean	true_soc_violation_rate_ci_low	true_soc_violation_r
cvar_interval	0.255754	0.174206	
dc3s_ftit	0.000000	0.000000	
dc3s_wrapped	0.000000	0.000000	
deterministic_lp	0.039087	0.029960	
robust_fixed_interval	0.255754	0.174206	

Risk: limited transferability beyond locked DE/US windows. Mitigation: explicit scope boundaries and future multi-region validation.

10.3 Cost–safety frontier (tradeoff curve)

Risk: proxy price/carbon signals may not represent all market settlement details. Mitigation: transparent assumptions and conservative interpretation.

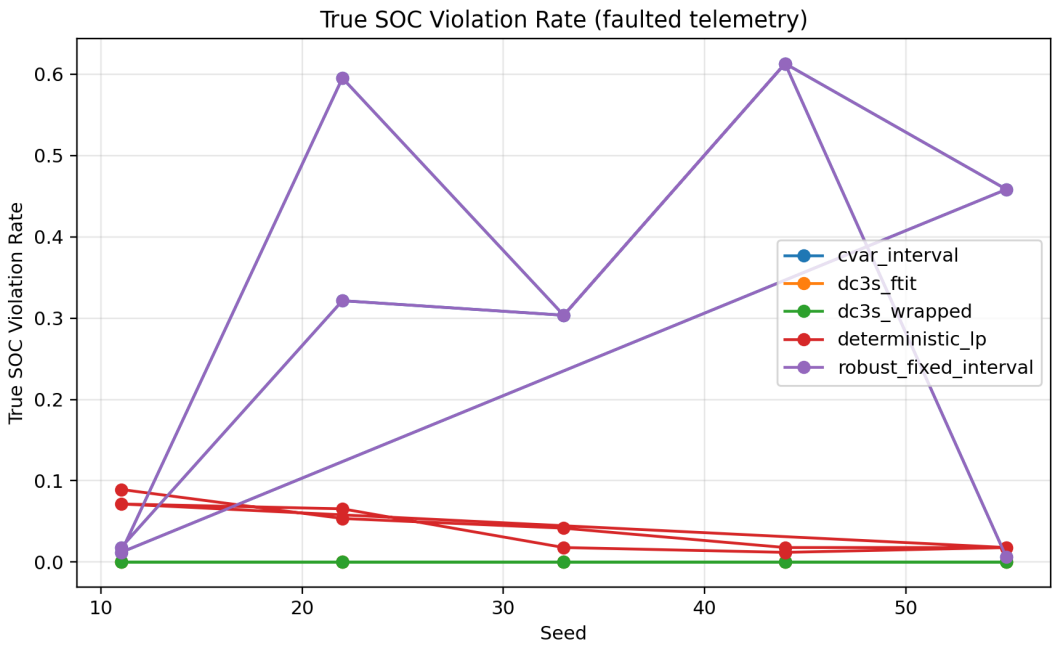

Figure 4: True SOC violation rate under fault severity sweep.

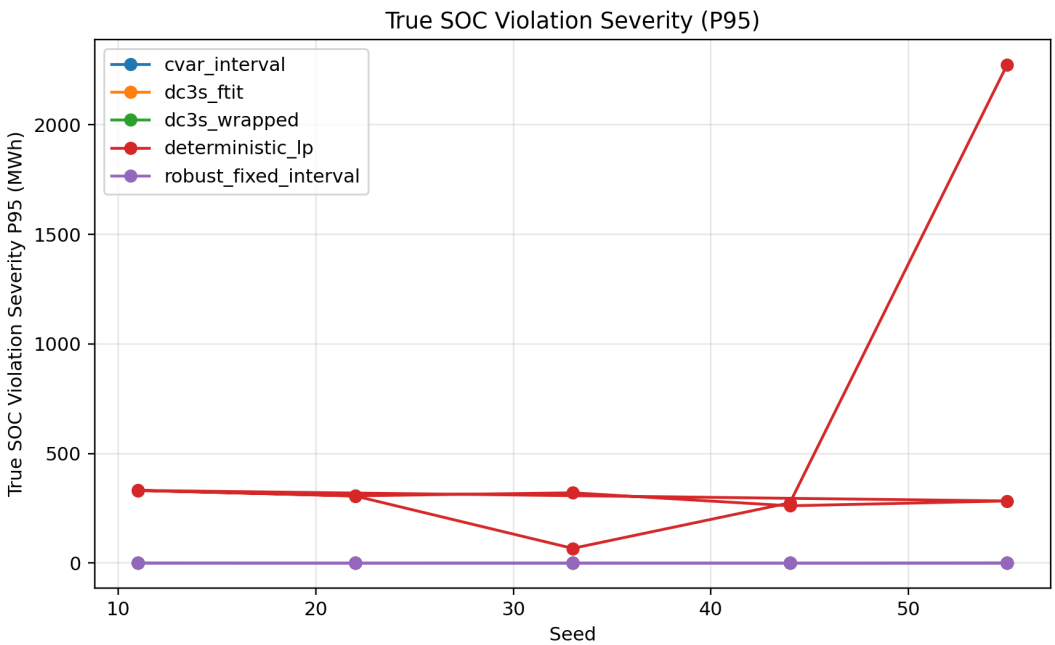

Figure 5: True SOC violation severity (P95) under fault severity sweep.

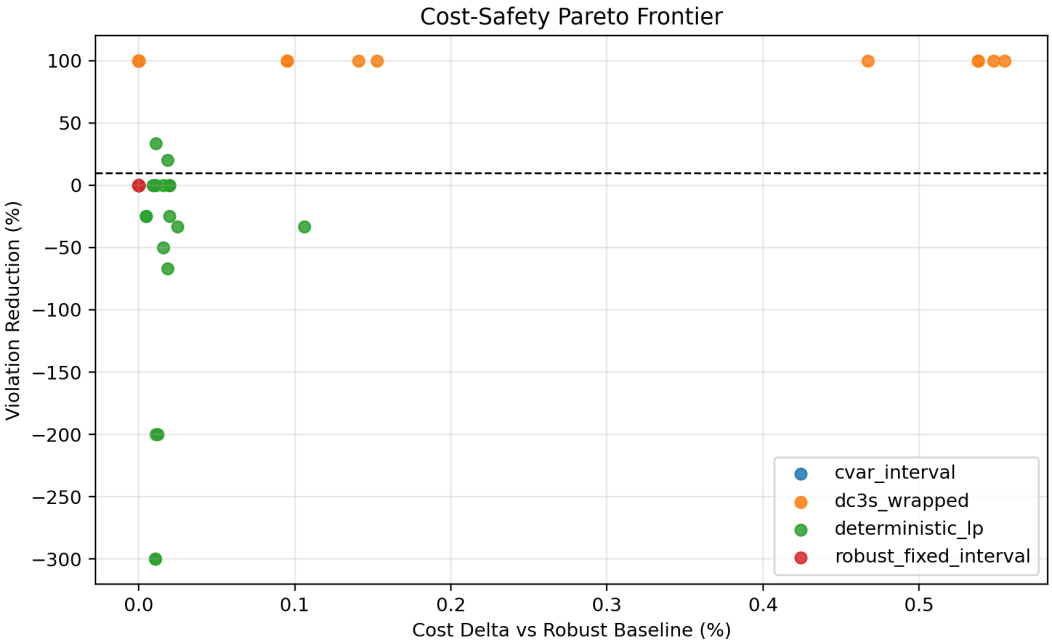

Figure 6: Cost-safety Pareto frontier from publication artifact build.

Table 23: Ablation study with strict gate fields and statistical tests.

analysis_scope	baseline_role	scenario	fault_dimension	baseline_controller
primary_aggregate_fault_sweep	primary	aggregate_fault_sweep	all	robust_fixed_interva
secondary_fault_dimension	secondary	aggregate_fault_sweep	delay_jitter	robust_fixed_interva
secondary_fault_dimension	secondary	aggregate_fault_sweep	dropout	robust_fixed_interva
secondary_fault_dimension	secondary	aggregate_fault_sweep	out_of_order	robust_fixed_interva
secondary_fault_dimension	secondary	aggregate_fault_sweep	spikes	robust_fixed_interva
secondary_scenario	secondary	drift_combo	aggregate	deterministic_lp
secondary_scenario	secondary	drift_combo	aggregate	robust_fixed_interva
secondary_scenario	secondary	dropout	aggregate	deterministic_lp
secondary_scenario	secondary	dropout	aggregate	robust_fixed_interva

10.4 Ablation study (what matters)

Risk: stochastic value magnitudes are sensitive to run/scenario settings. Mitigation: run-ID pinning and explicit source traceability.

10.5 Transfer/generalization results

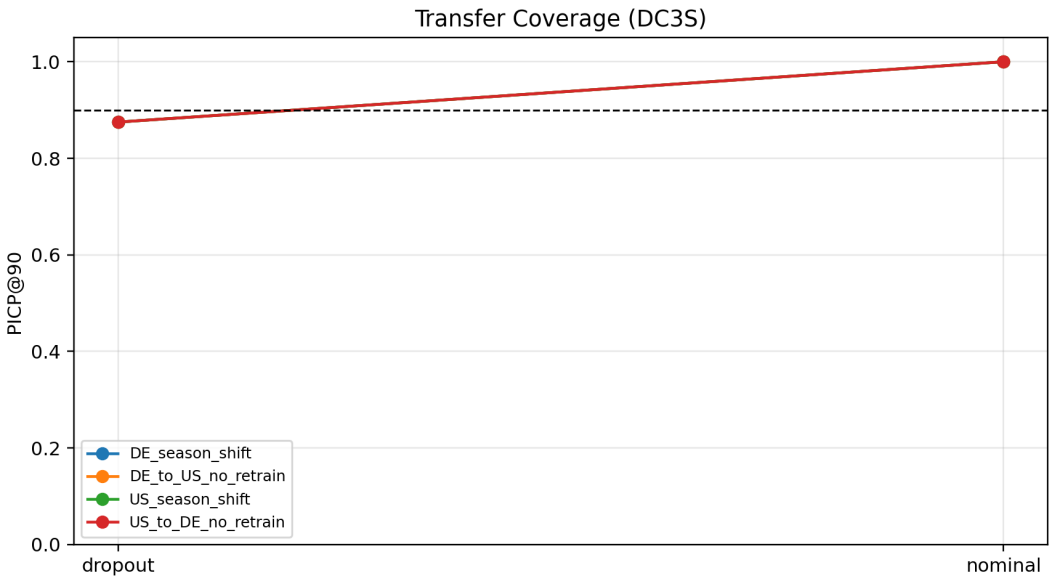

Figure 7: Transfer stress coverage summary across cross-region and seasonal cases.

1. Preserve strict metric synchronization across markdown, LaTeX, and DOCX.
2. Improve calibration on under-covered targets.
3. Expand external validity with additional regions/time windows.
4. Close citation gaps for non-repository claims.

10.6 Runtime overhead and intervention behavior

Runtime overhead is summarized through solver status, intervention frequency, and certificate completeness under faulted telemetry. Interpretation is comparative across controllers because absolute wall-clock latency depends on deployment hardware and solver backend details.

11 Discussion, Limitations, and Conclusion

11.1 Main takeaways (1 paragraph each)

11.2 Implications for CPS/AI safety

1. Detect (health, monitor, solver status).
2. Contain (block unsafe dispatch, switch fallback mode).
3. Diagnose (trace artifact/run lineage).
4. Recover (retrain, reconfigure, or rollback).
5. Record (update governance artifacts and incident notes).

Table 24: Transfer stress outcomes across DE/US and seasonal shifts.

transfer_case	sweep_type	sweep_value	scenario	seed	controller	picp_90	mea
DE_to_US_no_retrain	nominal	0.0	nominal	0	deterministic_lp	1.000000	65
DE_to_US_no_retrain	nominal	0.0	nominal	0	robust_fixed_interval	1.000000	65
DE_to_US_no_retrain	nominal	0.0	nominal	0	cvar_interval	1.000000	65
DE_to_US_no_retrain	nominal	0.0	nominal	0	dc3s_wrapped	1.000000	71
DE_to_US_no_retrain	nominal	0.0	nominal	1	deterministic_lp	1.000000	65
DE_to_US_no_retrain	nominal	0.0	nominal	1	robust_fixed_interval	1.000000	65
DE_to_US_no_retrain	nominal	0.0	nominal	1	cvar_interval	1.000000	65
DE_to_US_no_retrain	nominal	0.0	nominal	1	dc3s_wrapped	1.000000	71
DE_to_US_no_retrain	dropout	0.0	dropout	0	deterministic_lp	1.000000	65
DE_to_US_no_retrain	dropout	0.0	dropout	0	robust_fixed_interval	1.000000	65
DE_to_US_no_retrain	dropout	0.0	dropout	0	cvar_interval	1.000000	65
DE_to_US_no_retrain	dropout	0.0	dropout	0	dc3s_wrapped	1.000000	71
DE_to_US_no_retrain	dropout	0.0	dropout	1	deterministic_lp	1.000000	65
DE_to_US_no_retrain	dropout	0.0	dropout	1	robust_fixed_interval	1.000000	65
DE_to_US_no_retrain	dropout	0.0	dropout	1	cvar_interval	1.000000	65
DE_to_US_no_retrain	dropout	0.0	dropout	1	dc3s_wrapped	1.000000	71
DE_to_US_no_retrain	dropout	0.1	dropout	0	deterministic_lp	0.875000	65
DE_to_US_no_retrain	dropout	0.1	dropout	0	robust_fixed_interval	0.875000	65
DE_to_US_no_retrain	dropout	0.1	dropout	0	cvar_interval	0.875000	65
DE_to_US_no_retrain	dropout	0.1	dropout	0	dc3s_wrapped	0.875000	71
DE_to_US_no_retrain	dropout	0.1	dropout	1	deterministic_lp	0.875000	65
DE_to_US_no_retrain	dropout	0.1	dropout	1	robust_fixed_interval	0.875000	65
DE_to_US_no_retrain	dropout	0.1	dropout	1	cvar_interval	0.875000	65
DE_to_US_no_retrain	dropout	0.1	dropout	1	dc3s_wrapped	0.875000	71
DE_to_US_no_retrain	dropout	0.2	dropout	0	deterministic_lp	0.875000	65
DE_to_US_no_retrain	dropout	0.2	dropout	0	robust_fixed_interval	0.875000	65
DE_to_US_no_retrain	dropout	0.2	dropout	0	cvar_interval	0.875000	65
DE_to_US_no_retrain	dropout	0.2	dropout	0	dc3s_wrapped	0.875000	71
DE_to_US_no_retrain	dropout	0.2	dropout	1	deterministic_lp	0.875000	65
DE_to_US_no_retrain	dropout	0.2	dropout	1	robust_fixed_interval	0.875000	65
DE_to_US_no_retrain	dropout	0.2	dropout	1	cvar_interval	0.875000	65
DE_to_US_no_retrain	dropout	0.2	dropout	1	dc3s_wrapped	0.875000	71
DE_to_US_no_retrain	dropout	0.3	dropout	0	deterministic_lp	0.750000	65
DE_to_US_no_retrain	dropout	0.3	dropout	0	robust_fixed_interval	0.750000	65
DE_to_US_no_retrain	dropout	0.3	dropout	0	cvar_interval	0.750000	65
DE_to_US_no_retrain	dropout	0.3	dropout	0	dc3s_wrapped	0.750000	70
DE_to_US_no_retrain	dropout	0.3	dropout	1	deterministic_lp	0.750000	65
DE_to_US_no_retrain	dropout	0.3	dropout	1	robust_fixed_interval	0.750000	65
DE_to_US_no_retrain	dropout	0.3	dropout	1	cvar_interval	0.750000	65
DE_to_US_no_retrain	dropout	0.3	dropout	1	dc3s_wrapped	0.750000	71
DE_to_US_no_retrain	stale	0.0	nominal	0	deterministic_lp	1.000000	65
DE_to_US_no_retrain	stale	0.0	nominal	0	robust_fixed_interval	1.000000	65
DE_to_US_no_retrain	stale	0.0	nominal	0	cvar_interval	1.000000	65
DE_to_US_no_retrain	stale	0.0	nominal	0	dc3s_wrapped	1.000000	71
DE_to_US_no_retrain	stale	0.0	nominal	1	deterministic_lp	1.000000	65
DE_to_US_no_retrain	stale	0.0	nominal	1	robust_fixed_interval	1.000000	65
DE_to_US_no_retrain	stale	0.0	nominal	1	cvar_interval	1.000000	65
DE_to_US_no_retrain	stale	0.0	nominal	1	dc3s_wrapped	1.000000	71
DE_to_US_no_retrain	stale	0.1	nominal	0	deterministic_lp	1.000000	65
DE_to_US_no_retrain	stale	0.1	nominal	0	robust_fixed_interval	1.000000	65
DE_to_US_no_retrain	stale	0.1	nominal	0	cvar_interval	1.000000	65
DE_to_US_no_retrain	stale	0.1	nominal	0	dc3s_wrapped	1.000000	71
DE_to_US_no_retrain	stale	0.1	nominal	1	deterministic_lp	1.000000	65
DE_to_US_no_retrain	stale	0.1	nominal	1	robust_fixed_interval	1.000000	65
DE_to_US_no_retrain	stale	0.1	nominal	1	cvar_interval	1.000000	65
DE_to_US_no_retrain	stale	0.1	nominal	1	dc3s_wrapped	1.000000	71
DE_to_US_no_retrain	stale	0.2	nominal	0	deterministic_lp	1.000000	65
DE_to_US_no_retrain	stale	0.2	nominal	0	robust_fixed_interval	1.000000	65
DE_to_US_no_retrain	stale	0.2	nominal	0	cvar_interval	1.000000	65
DE_to_US_no_retrain	stale	0.2	nominal	0	dc3s_wrapped	1.000000	71
DE_to_US_no_retrain	stale	0.2	nominal	1	deterministic_lp	1.000000	65

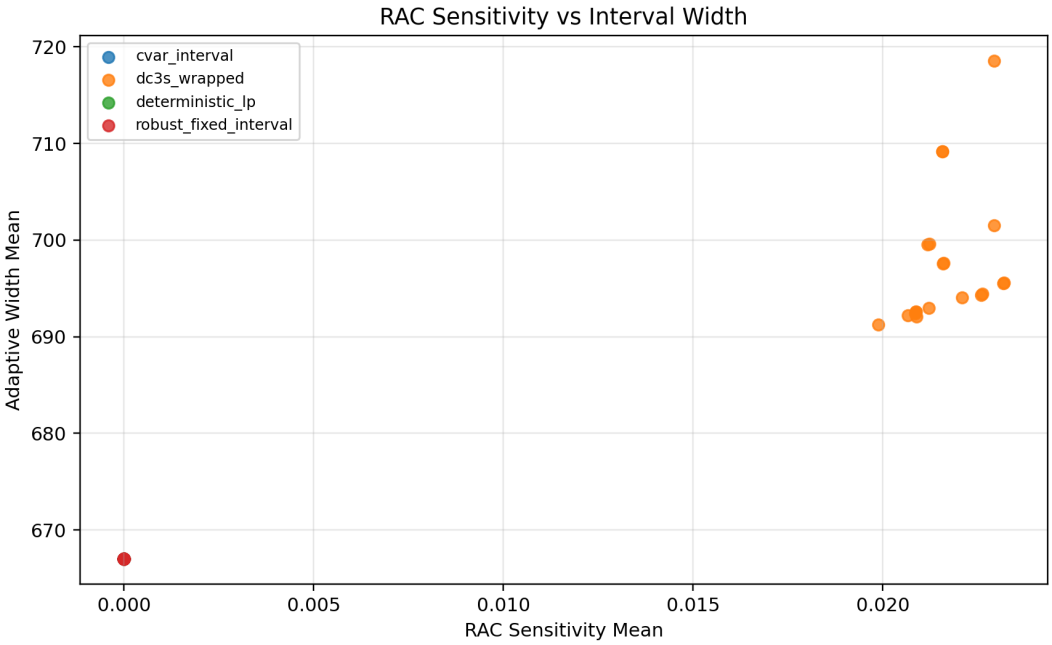

Figure 8: RAC-Cert sensitivity to interval-width relationship across controllers.

Table 25: Operational Failure Modes and Controls

Failure Mode	Trigger	Detection	Mitigation	Fallback
Missing/stale data	upstream delay or schema break	readiness checks and file checks	block unsafe requests until fresh inputs exist	health-warning state
Forecast drift	distributional shift	KS/model drift monitors	retraining decision workflow	conservative dispatch mode
Optimization infeasible	stress or malformed intervals	solver status and feasibility flags	safe infeasible response payload	grid-only baseline dispatch
Unsafe dispatch command	SOC/power violation	BMS validation	reject command with error detail	maintain current safe state
Control degradation	heartbeat timeout	watchdog island detection	lock remote control route	manual/local control
API auth misuse	invalid scope/key	security middleware	deny operation	read-only status routes

11.3 Governance Safeguards

1. Locked source-of-truth artifacts.
2. Validator-enforced banned legacy values.
3. Claim-level status and provenance tracking.

11.4 Config-Level Safety Snapshot

Selected defaults in locked configs:

1. KS drift threshold: `p_value_threshold = 0.01`.
2. Model drift threshold: `degradation_threshold = 0.15`.
3. Retraining cadence: 30 days with minimum new-data rule.
4. Deterministic objective weights: cost 1.0, carbon 1.2.
5. High unmet-load and curtailment penalties for safe optimization behavior.

11.5 Limitations (honest, non-defensive)

11.6 Dataset and Coverage Limits

Evidence is limited to the locked DE/US windows in this manuscript version. These limits are intentional for traceability and should only be expanded with corresponding manifest and claim-matrix updates.

11.7 Modeling and Optimization Limits

Stochastic value magnitudes are sensitive to scenario/stress choices and proxy assumptions. Robust objective weighting and interval-construction choices can shift tradeoff behavior, so results are evidence for this configured system rather than universal constants.

11.8 Documentation Synchronization Limits

Current shell limitations constrain programmatic DOCX synchronization:

- system `python3` lacks `python-docx`,
- local `.venv` launcher is currently unusable.

11.9 Citation Limits

Broader operational and societal claims require external source citation before publication. Claims without internal artifact evidence should remain explicitly marked as requiring citation.

11.10 IoT Validation Boundary (Deployment-Readiness, Non-Field Evidence)

IoT-related validation reported in this manuscript is software-in-the-loop/API/streaming-smoke evidence, not completed hardware field commissioning evidence.

1. Streaming checks rely on replayed telemetry and broker-consumer pipeline execution.
2. DC3S live checks are API-level evaluations driven by simulated/replayed telemetry inputs.

3. No physical inverter/BMS edge commissioning logs or field-trial artifacts are claimed in this revision.

Therefore, deployment-readiness claims are limited to software stack correctness and observability, not hardware-operational certification.

11.11 Roadmap to HIL/hardware validation

11.12 Near-Term

1. Unify report generation under one profile-lock workflow.
2. Add stricter claim-language linting for unsupported statements.
3. Improve interval calibration for under-covered targets.

Success condition: no unresolved claim conflicts and synchronized core metrics across markdown/LaTeX outputs.

11.13 Mid-Term

1. Extend stochastic scenario sensitivity studies.
2. Add more regions and tariff/carbon regimes.
3. Enhance model registry lineage and release metadata.

Success condition: cross-region comparisons with fully serialized scenario assumptions and reproducible run-level evidence.

11.14 Long-Term

1. Move to signed evidence bundles for release claims.
2. Integrate richer market and operational constraints.
3. Automate publication synchronization with deterministic release gates.

Success condition: signed provenance from data extraction to publication-ready manuscript artifacts.

11.15 Conclusion

GridPulse delivers a decision-grade ML system connecting forecasting, uncertainty, dispatch optimization, and governance. Under the dataset-scoped latest lock (February 17, 2026 artifacts), DE impact is 7.11% cost savings, 0.30% carbon reduction, and 6.13% peak shaving; US impact is 0.11% cost savings, 0.13% carbon reduction, and 0.00% peak shaving. Canonical stochastic values are tied to DE run `20260217_165756` and US run `20260217_182305`, with positive VSS in both regions (2,708.61 and 297,092.71). The thesis contribution is therefore both technical and operational: measurable decision outcomes plus reproducible claim governance. The practical message is that trustworthy decision systems require both technical performance and claim governance; GridPulse integrates forecast quality, uncertainty calibration, optimization feasibility, and publication traceability as one coherent system. Current IoT validation evidence in this manuscript is non-field and should be interpreted as software stack readiness rather than completed hardware-operational certification.

References

Internal Artifact References

- paper/metrics_manifest.json
- paper/claim_matrix.csv
- paper/accuracy_audit.md
- paper/rewrite_pack.md
- paper/sync_rules.md
- scripts/validate_paper_claims.py
- frontend/src/app/api/chat/tool-executors.ts
- services/api/routers/monitor.py
- src/gridpulse/streaming/run_consumer.py
- src/gridpulse/streaming/consumer.py
- src/gridpulse/streaming/worker.py
- configs/dc3s.yaml
- src/gridpulse/dc3s/
- services/api/routers/dc3s.py
- data/dashboard/manifest.json
- data/dashboard/de_stats.json
- data/dashboard/us_stats.json
- data/dashboard/de_metrics.json
- data/dashboard/us_metrics.json
- reports/impact_summary.csv
- reports/eia930/impact_summary.csv
- reports/impact_comparison.json
- reports/eia930/impact_comparison.json
- reports/research_metrics_de.csv
- reports/research_metrics_us.csv
- reports/publication/tables/table6_robustness.csv
- docs/ARCHITECTURE.md
- docs/TRAINING_PIPELINE.md
- docs/EVALUATION.md

External Method Literature (Maintain in Publication Export)

Conformal prediction, adaptive conformal inference, robust optimization, and time-series forecasting method references should be retained in the publication export.

External Factual Claims Requiring Citation

Any non-repository factual claim (policy forecasts, staffing claims, environmental lifecycle values) requires explicit external citation before publication.

A Appendix A: Algorithm listings (DC^{3}S + CQR calibration + CVaR)

1. Validate manuscript claims:

```
python scripts/validate_paper_claims.py
```

2. Compile LaTeX manuscript:

```
cd paper
pdflatex paper.tex
```

3. Confirm canonical strings exist in markdown and LaTeX:
   - `7.11%`
   - `0.11%`
   - `20260217_165756`
   - `20260217_182305`
   - `2,708.61`
   - `297,092.71`
4. Confirm no banned legacy percentages or placeholders.
5. Confirm publication numeric claims map to `paper/claim_matrix.csv`.

B Appendix B: Proof sketches / guarantee assumptions (short, clean)

C Appendix C: Hyperparameters + training configuration tables

This appendix defines the exact update protocol after retraining so tables/figures can be refreshed without introducing claim drift.

C.1 Update Philosophy

1. **Do not edit manuscript metrics manually first.**
2. Regenerate canonical artifacts using repository scripts.
3. Recompute claim tables from canonical artifacts.
4. Update manuscript values last, after validator pass.

Table 26: Core Governance and Evidence Artifacts

Artifact	Purpose
`paper/PAPER_DRAFT.md`	Master manuscript source
`paper/paper.tex`	LaTeX manuscript export
`paper/metrics_manifest.json`	Canonical metric lock and validation rules
`paper/claim_matrix.csv`	Claim-level provenance and status
`paper/accuracy_audit.md`	Accuracy/conflict audit log
`paper/rewrite_pack.md`	Detailed section guidance bank
`paper/sync_rules.md`	Markdown-to-LaTeX-to-DOCX synchronization contract
`scripts/validate_paper_claims.py`	Non-mutating cross-manuscript consistency checker
`data/dashboard/*`	Dataset profiles and dashboard metrics
`reports/impact_summary.csv`	Canonical DE impact outcomes
`reports/eia930/impact_summary.csv`	Canonical US impact outcomes
`reports/research_metrics_de.csv`	DE stochastic run outputs
`reports/research_metrics_us.csv`	US stochastic run outputs
`reports/publication/tables/table6_robustness.csv`	Perturbation robustness summary

Table 27: Core training and RAC-Cert hyperparameters.

group	key	value	source
train_forecast_de	horizon_hours	NaN	configs/train_forecast.yaml
train_forecast_de	lookback_hours	NaN	configs/train_forecast.yaml
train_forecast_de	cross_validation.n_folds	10	configs/train_forecast.yaml
train_forecast_de	cross_validation.gap	24	configs/train_forecast.yaml
train_forecast_us	horizon_hours	NaN	configs/train_forecast_eia930.yaml
train_forecast_us	lookback_hours	NaN	configs/train_forecast_eia930.yaml
train_forecast_us	cross_validation.n_folds	10	configs/train_forecast_eia930.yaml
train_forecast_us	cross_validation.gap	24	configs/train_forecast_eia930.yaml
dc3s_rac	sensitivity_probe	heuristic	configs/dc3s.yaml
dc3s_rac	sens_eps_mw	25.0	configs/dc3s.yaml
dc3s_rac	sens_norm_ref	0.5	configs/dc3s.yaml
dc3s_rac	beta_reliability	0.7	configs/dc3s.yaml
dc3s_rac	beta_sensitivity	0.5	configs/dc3s.yaml
dc3s_rac	k_sensitivity	0.4	configs/dc3s.yaml
regime_cqr	enabled	True	configs/uncertainty.yaml
regime_cqr	n_bins	3	configs/uncertainty.yaml
regime_cqr	vol_window	24	configs/uncertainty.yaml
regime_cqr	quantile_backend_policy	strict	configs/uncertainty.yaml

C.2 Canonical Build Order

1. Train/update uncertainty artifacts (Regime CQR + RAC).
2. Run CPSBench suite and fault sweeps.
3. Run ablations and strict statistical gate computation.
4. Run transfer stress and Pareto generation.
5. Run publication verifier and claim validator.

C.3 Operator Commands (Copy-Ready)

```
# 1) Train or refresh artifacts
python scripts/train_dataset.py --all --tune

# 2) Build publication package (canonical output path)
python scripts/build_publication_artifact.py \
  --out-dir reports/publication \
  --horizon 96 \
  --seeds 0 1 2 3 4 5 6 7 8 9

# 3) Verify artifact contract
python scripts/final_publish_audit.py --config configs/publish_audit.yaml
python scripts/validate_paper_claims.py
```

If runtime constraints require staged generation, run the same command at reduced horizon for correctness checks, then rerun with release horizon before final manuscript lock.

C.4 Figure/Table Replacement Map

Manuscript element	Canonical artifact source	Update rule
`Table2(Ablations)`	`reports/publication/table2_ablations.csv`	Replace entire table block, preserve metric names and gate flags.
`CQRgroupcoveragetable`	`reports/publication/cqr_group_coverage.csv`	Keep low/mid/high bins fixed; update values only.
`Transferstresstable`	`reports/publication/transfer_stress.csv` + `table5_transfer.csv`	Never use placeholder rows; include all 4 transfer cases.
`RACdiagnosticstext`	`reports/publication/rac_cert_summary.json`	Update hyperparameters and summary stats together.
`Safety-violationfigures`	`fig_true_soc_violation_vs_dropout.png`, `fig_true_soc_severity_p95_vs_dropout.png`	Replace both in same commit to avoid split evidence.
`Paretodiscussion`	`reports/publication/cost_safety_pareto.csv` + `fig_cost_safety_pareto.png`	Update cost/safety narrative from latest frontier points.
`Releasemanifestnote`	`reports/publication/release_manifest.json`	Confirm SHA/hash list and command lineage before submission.

C.5 Claim-Lock Checklist Before PDF Export

1. `stats_summary.json` shows `overall_pass=true`.
2. `primary_gate.passes_all_thresholds=true`.
3. Robust-baseline comparison satisfies $p < 0.01$ and target relative reduction.
4. `stats_summary.json` contains no temporary paths (`/tmp`).
5. Required artifacts are non-empty and hash-tracked in `release_manifest.json`.
6. `scripts/validate_paper_claims.py` exits successfully.

D Appendix D: Additional plots (all severities, per-region breakdown)

D.1 Hyperparameter and Config Disclosure Requirements

For final submission, disclose at minimum:

1. Forecast horizon, lookback, split ratios, and gap hours.
2. Conformal alpha, volatility binning window, and shrinkage τ.
3. Reliability, drift, and sensitivity inflation coefficients.
4. Robust/CVaR solver mode, scenario counts, and seeds.
5. Gate thresholds and bootstrap/Wilcoxon settings.

D.2 Ablation Matrix Minimum Rows

At minimum, report:

1. `deterministic_lp`
2. `robust_fixed_interval`
3. `cvar_interval`
4. `dc3s_wrapped (RAC-Cert path)`

Primary claim row must remain robust baseline vs DC3S candidate under faulted aggregate sweep.

D.3 Failure Analysis Requirements

Reviewer-facing failure analysis must include:

1. Scenarios where robust baseline outperforms candidate.
2. Cases with significance pass but effect-size miss.
3. Cases with effect-size pass but significance miss.
4. Any solver infeasibility rate changes after RAC updates.
5. Any transfer-case regression despite in-region gains.

D.4 Reproducibility Metadata Block

Every release PDF should include:

- Git commit SHA.
- Branch/tag name.
- Build command.
- Seed list and scenario list.
- Build timestamp and artifact hash manifest path.

This block can be inserted in the appendix as a table or literal JSON snippet.

D.5 What to Update Immediately After New Training

After fresh training or retuning, update these narrative zones first:

1. Abstract quantitative claim block.
2. Main results section percentages and confidence intervals.
3. Gate summary paragraph (pass/fail, metric-wise).
4. Transfer/generalization paragraph.
5. Limitations section if any gate fails or sensitivity changes substantially.

Do not update conclusion claims before these sections are synchronized.

E Appendix E: Reproducibility checklist (one command → all artifacts)

E.1 Introduction

Required facts:

1. Forecast-only systems are insufficient for operations.
2. GridPulse decision-loop framing.
3. Evidence-lock policy and date.

Optional additions: deployment context and operational motivation.

E.2 Data Assets and Scope

Required facts:

1. DE and US rows, date ranges, and feature counts.
2. Distinction between raw columns and engineered features.
3. Legacy-scope exclusion rule.

Required tables: dataset profile and feature family composition.

E.3 Methods

Required facts:

1. Forecast model families and horizon setup.
2. Conformal and adaptive interval logic.
3. Deterministic and robust optimization objectives/constraints.
4. Monitoring and safety implementation controls.

E.4 Results

Required facts:

1. Forecast metrics by model and target.
2. Coverage and residual diagnostics.
3. Canonical impact percentages and absolute outcomes.
4. Canonical stochastic values with run IDs.
5. Robustness perturbation summary.

E.5 Governance Sections

Required facts:

1. Source-of-truth artifact map.
2. Claim status model and editorial rule.
3. Validation gates and synchronization workflow.

E.6 Safety, Validity, and Limitations

Required facts:

1. Operational failure modes and mitigations.
2. Internal/external/construct/conclusion validity boundaries.
3. Current environment limitations for DOCX sync.

E.7 Claims Requiring Citation Before Publication

The following claim classes require external source citation or removal in publication export:

1. Global market-size or policy forecasts.
2. Named-operator staffing claims.
3. Environmental footprint values without reproducible in-repo method.
4. Universal superiority statements beyond locked DE/US evidence.

E.8 Synchronization and Release Notes

1. `paper/PAPER_DRAFT.md` remains the authority manuscript.
2. `paper/paper.tex` must preserve identical title, abstract core metrics, and conclusion core metrics.
3. DOCX synchronization is performed last and must record tooling limits when automation is unavailable.
4. Final release gate requires validator pass and no unresolved publication-facing claim conflicts.

E.9 Full-Detail Authoring Blocks (Use/Trim as Needed)

E.10 Long-Form Intro Block

GridPulse is designed around a decision-first interpretation of machine learning in energy systems. Rather than ending at point prediction, the system carries uncertainty information into optimization, enforces dispatch feasibility under battery and grid constraints, and evaluates outcomes against explicit baselines. This architecture creates a measurable bridge between model behavior and operational value.

E.11 Long-Form Method Block

The forecasting layer trains LightGBM, LSTM, and TCN families under time-aware configurations with a 24-hour horizon and 168-hour lookback. Uncertainty is represented using conformal intervals with adaptive behavior. Deterministic optimization uses a mixed-integer structure with cost, carbon, degradation, curtailment, unmet-load, and peak terms. Robust optimization uses a two-scenario epigraph formulation over lower and upper load bounds, and safety checks are enforced before dispatch control actions are accepted.

E.12 Long-Form Results Block

Under the locked artifact policy, DE impact is 7.11% cost savings, 0.30% carbon reduction, and 6.13% peak shaving. US impact is 0.11% cost savings, 0.13% carbon reduction, and 0.00% peak shaving. Canonical stochastic evidence is tied to DE run `20260217_165756` and US run `20260217_182305`; both runs are feasible in run-summary outputs. Positive VSS appears in both regions with substantially different magnitudes.

E.13 Long-Form Governance Block

The manuscript uses a source-of-truth contract where publication-facing numeric claims must map to `paper/metrics_manifest.json` and corresponding entries in `paper/claim_matrix.csv`. Validation is automated via `scripts/validate_paper_claims.py`, which enforces required canonical strings and blocks known legacy contradictions.

E.14 Long-Form Limitation Block

Evidence is intentionally restricted to locked DE and US windows from dashboard profile artifacts. This improves traceability but limits direct external generalization. Optimization outcomes also depend on proxy price/carbon assumptions and scenario/interval settings that may vary across market designs.

E.15 Long-Form Future Work Block

Future work should unify report-generation pathways, improve uncertainty calibration where coverage is below nominal, expand region and tariff coverage, and strengthen release automation with signed evidence bundles.

E.16 Keep/Trim Guidance for Final Submission

If page limits require reduction:

1. Keep Sections 6, 7, 8, and 14 unchanged.
2. Keep all canonical run IDs and percentages unchanged.
3. Trim descriptive implementation prose before trimming governance controls.
4. Preserve limitations and citation-boundary statements.

E.17 Project-to-Paper Coverage Map

This appendix maps implemented project surfaces to manuscript coverage so review can distinguish fully documented areas from areas that are currently summarized.

Coverage status definitions:

1. **Full**: implementation paths and operational behavior are explicitly described in the paper.
2. **Partial**: core behavior is described, but implementation/runtime details are summarized.

Table 29: Project-to-Paper Coverage Matrix

Project Surface	Primary Implementation Paths	Current Paper Coverage	Status	Notes
Forecast training orchestration, splits, tuning	`src/gridpulse/forecasting/train.py`, `configs/train_forecast.yaml`, `configs/train_forecast_eia930.yaml`, `scripts/train_dataset.py`	Sections 3.7, 4.1	Full	Runtime control flags (`-tune`, `-no-tune`, `-ensemble`, `-max-seeds`, `-n-trials`, `-top-pct`) plus tuning/ensemble serialization behavior are explicitly documented.
Forecast inference and bundle resolution	`services/api/routers/forecast.py`, `src/gridpulse/forecasting/predict.py`	Sections 2.3, 4.1.4	Full	Explicit target-path resolution, ordered fallback search, ensemble averaging, quantile serving order, and missing-target signaling are documented.
Streaming ingestion and persistence	`src/gridpulse/streaming/run_consumer.py`, `src/gridpulse/streaming/consumer.py`, `src/gridpulse/streaming/worker.py`, `configs/streaming.yaml`	Sections 2.7, 12.5	Full	CLI path, config load path, and DuckDB table contract are explicitly documented.
DC3S uncertainty shield and audit chain	`src/gridpulse/dc3s/`, `services/api/routers/dc3s.py`, `configs/dc3s.yaml`	Sections 2.3, 2.7, 4.5, 12.5	Full	Endpoint contracts, implemented inflation law, shield modes, hash chain, and state/audit behavior are documented.
Monitoring and retraining	`services/api/routers/monitor.py`, `src/gridpulse/monitoring/`, `configs/monitoring.yaml`	Section 4.6	Full	Drift thresholds, cadence, and decision logic are explicitly documented.
Safety and control-plane enforcement	`src/gridpulse/safety/`, `services/api/main.py` (`/control/dispatch`)	Sections 4.7, 11	Full	Authorization, watchdog, and BMS gate sequence is documented.
Frontend operator workflow and live DC3S UX	`frontend/src/app/(dashboard)/page.tsx`, `frontend/src/components/dashboard/DC3SLiveCard.tsx`, `frontend/src/lib/api/dc3s-client.ts`, `frontend/src/app/api/dc3s/audit/[commandId]/route.ts`	Sections 2.2, 2.7	Full	Dashboard polling default, selectable auto-refresh cadence, manual refresh, command-linked audit quick link, and proxy/error behavior are explicitly documented.
Claim governance and release gates	`paper/metrics_manifest.json`, `paper/claim_matrix.csv`, `scripts/validate_paper_claims.py`	Sections 5, 7, 8	Full	Source lock, reconciliation, and publication gating are explicitly documented.

E.18 Full-Coverage Update (Completed)

This revision closes the previously partial coverage areas:

1. Section 4.1 now documents runtime training controls and their reproducibility implications.
2. Section 4.1.4 now documents fallback/ensemble/quantile inference resolution behavior.
3. Section 2.7 now documents frontend live-ops controls and command-linked audit navigation.
4. Baseline policy definitions and impact-table numerator/denominator mapping are now explicit for B1/B2/B3.